

Creating Star Schema

This notebook explores the creation of a Star Schema from raw data using Apache Spark.

The data used in this notebook is from the Brazilian Basic Education Census, which is available at the [Brazilian government's open data portal](#)

Creating SparkSession

The first step is to create a SparkSession.

```
In [1]: #1
# import os
# os.environ['HADOOP_HOME'] = 'C:\\hadoop'
# os.environ['PYSPARK_PYTHON'] = 'C:\\Python311\\python.exe'
# os.environ['PYSPARK_DRIVER_PYTHON'] = 'C:\\Python311\\python.exe'
# if 'SPARK_HOME' in os.environ:
#     del os.environ['SPARK_HOME']
# print("✅ Environment ready!")
import os

# Clear ALL spark-related environment variables
for key in ['SPARK_HOME', 'PYSPARK_SUBMIT_ARGS']:
    if key in os.environ:
        del os.environ[key]

# Set correct ones
os.environ['HADOOP_HOME'] = 'C:\\hadoop'
os.environ['PYSPARK_PYTHON'] = 'C:\\Python311\\python.exe'
os.environ['PYSPARK_DRIVER_PYTHON'] = 'C:\\Python311\\python.exe'

print("✅ Environment ready!")
```

✅ Environment ready!

```
In [2]: #2
from pyspark.sql import SparkSession
import pyspark.sql.functions as F

import psycpg2
from datetime import datetime

from itertools import chain
print("✅ Libraries imported!")
```

✅ Libraries imported!

```
In [3]: #3
#JDBC driver path
spark = SparkSession.builder \
    .appName("CensoEscolarStarSchema") \
    .config("spark.sql.shuffle.partitions", "4") \
    .config("spark.driver.memory", "4g") \
    .master("local[*]") \
    .getOrCreate()
```

```
print("✅ Spark ready! Version:", spark.version)
```

✅ Spark ready! Version: 3.5.1

```
In [4]: # Check if JAR exists
import os
jar_path = "C:\\Users\\HUAWEI\\ApacheSparkWork\\posts\\etl_spark_scholar_census\\"
if os.path.exists(jar_path):
    print(f"✅ JAR found: {jar_path}")
else:
    print(f"❌ JAR NOT found at: {jar_path}")
    print("Please download postgresql-42.7.3.jar and place it in the project fol
```

✅ JAR found: C:\Users\HUAWEI\ApacheSparkWork\posts\etl_spark_scholar_census\postgresql-42.7.3.jar

Transforming CSV to Parquet

This transformation aims to increase the speed of data loading by using Parquet files.

```
In [5]: # Read CSV data
data_csv = (
    spark
    .read
    .format("csv")
    .option("header", "true")
    .option("inferSchema", "false")
    .option("delimiter", ";")
    .option("encoding", "latin1")
    .load("./data/*.csv")
)
print(f"✅ CSV loaded! Columns: {len(data_csv.columns)}")
```

✅ CSV loaded! Columns: 370

Write to Parquet file

```
In [6]: # Write to Parquet file
data_csv.write.mode("overwrite").parquet("./data/censo_escolar.parquet")
print("✅ Parquet file written successfully!")
```

✅ Parquet file written successfully!

Reading from Parquet file

```
In [27]: #4
#Reading from Parquet file
data = (
    spark
    .read
    .format("parquet")
    .load("./data/censo_escolar.parquet/")
)

print("✅ Parquet loaded successfully!")
print(f"Total rows: {data.count()}")
print(f"Total columns: {len(data.columns)}")
```

✅ Parquet loaded successfully!

Total rows: 2792984

Total columns: 370

Dimensions

The dimensions are...

The code below creates the dimensions based on a configuration dict.

```
{
  "DIM_NAME":{
    # The fields are the table columns
    "fields":[
      {
        "field":"FIELD_1_NAME", # The column name
        "type":"FIELD_1_TYPE", # The column type in spark
      },
      {
        "field":"FIELD_2_NAME",
        "type":"FIELD_2_TYPE",
      },
      ...
    ],
  }
}
```

```
In [12]: #5
INTEGER_DIMENSIONS = [
    "TP_DEPENDENCIA",
    "TP_LOCALIZACAO",
    "IN_AGUA_POTAVEL",
    "IN_ENERGIA_INEXISTENTE",
    "IN_ESGOTO_INEXISTENTE",
    "IN_BANHEIRO",
    "IN_BIBLIOTECA",
    "IN_REFEITORIO",
    "IN_COMPUTADOR",
    "IN_INTERNET",
    "IN_EQUIP_NENHUM"
]

DIMENSION_TABLES_CONFIG = {
    "DIM_LOCAL":{
        "fields": [
            {"field":"NO_UF", "type":"string",},
            {"field":"SG_UF", "type":"string",},
            {"field":"CO_UF", "type":"string",},
            {"field":"NO_MUNICIPIO", "type":"string",},
            {"field":"CO_MUNICIPIO", "type":"string",}
        ]
    },
}

DIMENSION_TABLES_CONFIG.update(
    {
```

```

        "DIM_"+dimension.upper():{
            "fields": [
                {"field":dimension, "type":"integer"}
            ]
        }
        for dimension in INTEGER_DIMENSIONS
    }
)

print("✅ Dimension config ready!")
print(f"Total dimensions: {len(DIMENSION_TABLES_CONFIG)}")

```

✅ Dimension config ready!
Total dimensions: 12

Creating dimensions table in Postgres

Defining the properties of the Postgres Connection

```

In [13]: #6
POSTGRES_USER = "censo"
POSTGRES_PASSWORD = "123"
POSTGRES_DB = "censo_escolar"

# Used to connect to the PostgreSQL database server
# in spark session
POSTGRES_CONFIG = {
    "url":f"jdbc:postgresql://localhost:5432/{POSTGRES_DB}",
    "properties":{
        "user":POSTGRES_USER,
        "password":POSTGRES_PASSWORD,
        "driver":"org.postgresql.Driver",
    },
}

```

Establishing connection to Postgres

```

In [14]: #8
conn = psycopg2.connect(
    host="127.0.0.1", # ← changed from localhost
    port=5432, # ← integer not string
    dbname=POSTGRES_DB,
    user=POSTGRES_USER,
    password=POSTGRES_PASSWORD
)
print("✅ PostgreSQL connected!")

```

✅ PostgreSQL connected!

```

In [17]: # Drop all tables in correct order (fact first, then dimensions)
cursor = conn.cursor()
try:
    cursor.execute("""
        DROP TABLE IF EXISTS fact_censo_escolar CASCADE;
        DROP TABLE IF EXISTS dim_local CASCADE;
        DROP TABLE IF EXISTS dim_tp_dependencia CASCADE;
        DROP TABLE IF EXISTS dim_tp_localizacao CASCADE;
        DROP TABLE IF EXISTS dim_in_agua_potavel CASCADE;
    """)

```

```

DROP TABLE IF EXISTS dim_in_energia_inexistente CASCADE;
DROP TABLE IF EXISTS dim_in_esgoto_inexistente CASCADE;
DROP TABLE IF EXISTS dim_in_banheiro CASCADE;
DROP TABLE IF EXISTS dim_in_biblioteca CASCADE;
DROP TABLE IF EXISTS dim_in_refeitorio CASCADE;
DROP TABLE IF EXISTS dim_in_computador CASCADE;
DROP TABLE IF EXISTS dim_in_internet CASCADE;
DROP TABLE IF EXISTS dim_in_equip_nenhum CASCADE;
"""
)
conn.commit()
print("✅ All tables dropped!")
except Exception as e:
    print(f"❌ Error: {e}")
    conn.rollback()
finally:
    cursor.close()

```

✅ All tables dropped!

Function to create a Dimension table in Postgres using the configuration in DIMENSION_TABLES_CONFIG and adding an id column

The code below creates the dimensions Spark will create a table with the name of the dimension and the columns in the configuration

```

In [18]: # Write data to Postgres
# Using the configuration in DIMENSION_TABLES_CONFIG
# With id as the primary key

for table_name, table_config in DIMENSION_TABLES_CONFIG.items():
    print(f"[{datetime.now()}] Writing {table_name}")

    data\
    .select([
        F.col(field["field"])
        .cast(field["type"])
        .alias(field["field"])
        for field in table_config["fields"]
    ])\
    .distinct()\
    .withColumn("id", F.monotonically_increasing_id())\
    .write\
    .jdbc(
        **POSTGRES_CONFIG,
        table=table_name,
        mode="overwrite"
    )

    print(f"[{datetime.now()}] Wrote {table_name}")
    cursor = conn.cursor()
    cursor.execute(f"ALTER TABLE {table_name} ADD PRIMARY KEY (id);")
    cursor.close()
    conn.commit()
    print(f"[{datetime.now()}] Done {table_name} ✅")

print("\n✅ All dimension tables written!")

```

```

[2026-05-12 21:30:26.360317] Writing DIM_LOCAL
[2026-05-12 21:30:30.311447] Wrote DIM_LOCAL
[2026-05-12 21:30:30.359522] Done DIM_LOCAL ✓
[2026-05-12 21:30:30.360053] Writing DIM_TP_DEPENDENCIA
[2026-05-12 21:30:31.895606] Wrote DIM_TP_DEPENDENCIA
[2026-05-12 21:30:31.909413] Done DIM_TP_DEPENDENCIA ✓
[2026-05-12 21:30:31.909413] Writing DIM_TP_LOCALIZACAO
[2026-05-12 21:30:32.680437] Wrote DIM_TP_LOCALIZACAO
[2026-05-12 21:30:32.694399] Done DIM_TP_LOCALIZACAO ✓
[2026-05-12 21:30:32.694399] Writing DIM_IN_AGUA_POTAVEL
[2026-05-12 21:30:33.584865] Wrote DIM_IN_AGUA_POTAVEL
[2026-05-12 21:30:33.603649] Done DIM_IN_AGUA_POTAVEL ✓
[2026-05-12 21:30:33.603649] Writing DIM_IN_ENERGIA_INEXISTENTE
[2026-05-12 21:30:34.501581] Wrote DIM_IN_ENERGIA_INEXISTENTE
[2026-05-12 21:30:34.516568] Done DIM_IN_ENERGIA_INEXISTENTE ✓
[2026-05-12 21:30:34.517076] Writing DIM_IN_ESGOTO_INEXISTENTE
[2026-05-12 21:30:35.287307] Wrote DIM_IN_ESGOTO_INEXISTENTE
[2026-05-12 21:30:35.300940] Done DIM_IN_ESGOTO_INEXISTENTE ✓
[2026-05-12 21:30:35.300969] Writing DIM_IN_BANHEIRO
[2026-05-12 21:30:35.993247] Wrote DIM_IN_BANHEIRO
[2026-05-12 21:30:36.005372] Done DIM_IN_BANHEIRO ✓
[2026-05-12 21:30:36.005372] Writing DIM_IN_BIBLIOTECA
[2026-05-12 21:30:36.708578] Wrote DIM_IN_BIBLIOTECA
[2026-05-12 21:30:36.722188] Done DIM_IN_BIBLIOTECA ✓
[2026-05-12 21:30:36.722188] Writing DIM_IN_REFEITORIO
[2026-05-12 21:30:37.388599] Wrote DIM_IN_REFEITORIO
[2026-05-12 21:30:37.401575] Done DIM_IN_REFEITORIO ✓
[2026-05-12 21:30:37.401575] Writing DIM_IN_COMPUTADOR
[2026-05-12 21:30:38.064228] Wrote DIM_IN_COMPUTADOR
[2026-05-12 21:30:38.076922] Done DIM_IN_COMPUTADOR ✓
[2026-05-12 21:30:38.076922] Writing DIM_IN_INTERNET
[2026-05-12 21:30:38.741145] Wrote DIM_IN_INTERNET
[2026-05-12 21:30:38.754960] Done DIM_IN_INTERNET ✓
[2026-05-12 21:30:38.755468] Writing DIM_IN_EQUIP_NENHUM
[2026-05-12 21:30:39.406500] Wrote DIM_IN_EQUIP_NENHUM
[2026-05-12 21:30:39.418593] Done DIM_IN_EQUIP_NENHUM ✓

```

✓ All dimension tables written!

Facts table

The definition of the facts table follows a different pattern than the dimensions.

The table schema is previously defined to properly define the foreign keys.

Defining the facts table schema Metrics + Facts + Dimensions (Foreign Keys)

In [19]: `FACT_TABLE_NAME = "FACT_CENSO_ESCOLAR"`

```

FACT_COLUMNS = [
    "QT_DOC_BAS",
    "QT_DOC_INF",
    "QT_DOC_FUND",
    "QT_DOC_MED",
    "QT_MAT_BAS",
    "QT_MAT_INF",
    "QT_MAT_FUND",
    "QT_MAT_MED",

```

```

    "QT_MAT_BAS_ND",
    "QT_MAT_BAS_BRANCA",
    "QT_MAT_BAS_PRETA",
    "QT_MAT_BAS_PARDA",
    "QT_MAT_BAS_AMARELA",
    "QT_MAT_BAS_INDIGENA",
    "NU_ANO_CENSO"
]

FACT_CONFIG = {
    fact:{
        "fields": [
            {"field":fact, "type":"integer"}
        ]
    }
    for fact in FACT_COLUMNS
}

DIMENSION_ID_CONFIG = {
    table_name:[
        field['field']
        for field
        in table_fields['fields']
    ]
    for table_name, table_fields in DIMENSION_TABLES_CONFIG.items()
}

FACT_TABLE_ALL_COLUMNS_ORDERED = FACT_COLUMNS + list(map(lambda col:"ID_"+col, D

print("✅ Fact config ready!")
print(f"Fact columns: {len(FACT_COLUMNS)}")
print(f"Dimension IDs: {len(DIMENSION_ID_CONFIG)}")
print(f"Total ordered columns: {len(FACT_TABLE_ALL_COLUMNS_ORDERED)}")

```

✅ Fact config ready!

Fact columns: 15

Dimension IDs: 12

Total ordered columns: 27

Before inserting the data into the facts table, we need to create a function to create the facts table in Postgres

The code below creates the facts table in Postgres using the configuration in FACT_TABLES_CONFIG and adding an id column for each dimension

```

In [20]: # Create fact table
# Using the configuration in FACT_CONFIG
# With id as the primary key

# Avoid inserting a backslash into a f-string
comma_break_line = ",\n\t\t\t\t\t"

# Create table SQL
create_table_sql = f"""
    CREATE TABLE IF NOT EXISTS {FACT_TABLE_NAME} (
        id SERIAL PRIMARY KEY,
        {comma_break_line.join(
            [f"{field} INTEGER" for field in FACT_COLUMNS]
            + [f"ID_{dim_table} BIGINT" for dim_table in DIMENSION_ID_CONFIG.key

```

```

    })
    );
"""

# Foreign keys - each as separate ALTER TABLE statement
foreign_key_sqls = [
    f"""ALTER TABLE {FACT_TABLE_NAME}
    ADD CONSTRAINT {FACT_TABLE_NAME}_{dim_table}_fk
    FOREIGN KEY (ID_{dim_table})
    REFERENCES {dim_table}(id);"""
    for dim_table in DIMENSION_ID_CONFIG.keys()
]

print("✅ SQL generated!")
print(create_table_sql)

```

✅ SQL generated!

```

CREATE TABLE IF NOT EXISTS FACT_CENSO_ESCOLAR (
    id SERIAL PRIMARY KEY,
    QT_DOC_BAS INTEGER,
    QT_DOC_INF INTEGER,
    QT_DOC_FUND INTEGER,
    QT_DOC_MED INTEGER,
    QT_MAT_BAS INTEGER,
    QT_MAT_INF INTEGER,
    QT_MAT_FUND INTEGER,
    QT_MAT_MED INTEGER,
    QT_MAT_BAS_ND INTEGER,
    QT_MAT_BAS_BRANCA INTEGER,
    QT_MAT_BAS_PRETA INTEGER,
    QT_MAT_BAS_PARDA INTEGER,
    QT_MAT_BAS_AMARELA INTEGER,
    QT_MAT_BAS_INDIGENA INTEGER,
    NU_ANO_CENSO INTEGER,
    ID_DIM_LOCAL BIGINT,
    ID_DIM_TP_DEPENDENCIA BIGINT,
    ID_DIM_TP_LOCALIZACAO BIGINT,
    ID_DIM_IN_AGUA_POTAVEL BIGINT,
    ID_DIM_IN_ENERGIA_INEXISTENTE BIGINT,
    ID_DIM_IN_ESGOTO_INEXISTENTE BIGINT,
    ID_DIM_IN_BANHEIRO BIGINT,
    ID_DIM_IN_BIBLIOTECA BIGINT,
    ID_DIM_IN_REFEITORIO BIGINT,
    ID_DIM_IN_COMPUTADOR BIGINT,
    ID_DIM_IN_INTERNET BIGINT,
    ID_DIM_IN_EQUIP_NENHUM BIGINT
);

```

Executing the function to create the facts table in Postgres

```

In [22]: print(f"[{datetime.now()}] Creating facts table")

cursor = conn.cursor()
try:
    # Create the table first
    cursor.execute(create_table_sql)
    conn.commit()
    print(f"✅ Table created!")

```



```

# Add foreign keys one by one
for fk_sql in foreign_key_sqls:
    try:
        cursor.execute(fk_sql)
        conn.commit()
    except Exception as fk_error:
        print(f"⚠️ FK warning (may already exist): {fk_error}")
        conn.rollback()

print(f"[{datetime.now()}] Created facts table ✅")
print(f"[{datetime.now()}] Done")

except Exception as e:
    print(f"❌ Error: {e}")
    conn.rollback()
finally:
    cursor.close()

```

[2026-05-12 21:30:57.705744] Creating facts table

✅ Table created!

[2026-05-12 21:30:57.948586] Created facts table ✅

[2026-05-12 21:30:57.949133] Done

Extracting the data

In [23]:

```

# Extracting the data
facts_data = data\
    .select(
        [
            *chain(
                *DIMENSION_ID_CONFIG.values(),
                FACT_CONFIG.keys()
            )
        ]
    )

print("✅ Facts data extracted!")
print(f"Columns selected: {len(facts_data.columns)}")

```

✅ Facts data extracted!

Columns selected: 31

Adding the ids for each dimension

In [24]:

```

# Joining the id of the dimensions
print("Starting dimension joins...")

for table_name, table_fields in DIMENSION_ID_CONFIG.items():
    print(f"  Joining {table_name}...")

    dim_table = spark.read\
        .jdbc(
            **POSTGRES_CONFIG,
            table=table_name,
        )\
        .withColumnRenamed("id", f"ID_{table_name}")

    facts_data = facts_data\
        .join(

```

```

        dim_table,
        on=table_fields,
        how="left"
    )\
    .drop(*table_fields)

print("✅ All dimensions joined!")
print(f"Final columns: {len(facts_data.columns)}")

```

Starting dimension joins...

```

Joining DIM_LOCAL...
Joining DIM_TP_DEPENDENCIA...
Joining DIM_TP_LOCALIZACAO...
Joining DIM_IN_AGUA_POTAVEL...
Joining DIM_IN_ENERGIA_INEXISTENTE...
Joining DIM_IN_ESGOTO_INEXISTENTE...
Joining DIM_IN_BANHEIRO...
Joining DIM_IN_BIBLIOTECA...
Joining DIM_IN_REFEITORIO...
Joining DIM_IN_COMPUTADOR...
Joining DIM_IN_INTERNET...
Joining DIM_IN_EQUIP_NENHUM...

```

✅ All dimensions joined!

Final columns: 27

Saving data to Postgres

```

In [25]: # Order the columns to match the fact table on postgres
# and save the data
from pyspark.sql.functions import col

print(f"[{datetime.now()}] Saving facts data to PostgreSQL...")
print("⚠️ This will take 20-40 minutes for 11 years of data...")

# Cast all FACT_COLUMNS to integer before saving
facts_data_casted = facts_data.select(
    # Cast fact columns to integer
    *[col(c).cast("integer").alias(c) for c in FACT_COLUMNS],
    # Keep dimension ID columns as-is (already bigint)
    *[col(f"ID_{dim}") for dim in DIMENSION_ID_CONFIG.keys()]
)

print("✅ Columns casted!")
print(f"Schema sample:")
facts_data_casted.printSchema()

```

[2026-05-12 21:31:04.885610] Saving facts data to PostgreSQL...

⚠ This will take 20-40 minutes for 11 years of data...

✅ Columns casted!

Schema sample:

root

```
-- QT_DOC_BAS: integer (nullable = true)
-- QT_DOC_INF: integer (nullable = true)
-- QT_DOC_FUND: integer (nullable = true)
-- QT_DOC_MED: integer (nullable = true)
-- QT_MAT_BAS: integer (nullable = true)
-- QT_MAT_INF: integer (nullable = true)
-- QT_MAT_FUND: integer (nullable = true)
-- QT_MAT_MED: integer (nullable = true)
-- QT_MAT_BAS_ND: integer (nullable = true)
-- QT_MAT_BAS_BRANCA: integer (nullable = true)
-- QT_MAT_BAS_PRETA: integer (nullable = true)
-- QT_MAT_BAS_PARDA: integer (nullable = true)
-- QT_MAT_BAS_AMARELA: integer (nullable = true)
-- QT_MAT_BAS_INDIGENA: integer (nullable = true)
-- NU_ANO_CENSO: integer (nullable = true)
-- ID_DIM_LOCAL: long (nullable = true)
-- ID_DIM_TP_DEPENDENCIA: long (nullable = true)
-- ID_DIM_TP_LOCALIZACAO: long (nullable = true)
-- ID_DIM_IN_AGUA_POTAVEL: long (nullable = true)
-- ID_DIM_IN_ENERGIA_INEXISTENTE: long (nullable = true)
-- ID_DIM_IN_ESGOTO_INEXISTENTE: long (nullable = true)
-- ID_DIM_IN_BANHEIRO: long (nullable = true)
-- ID_DIM_IN_BIBLIOTECA: long (nullable = true)
-- ID_DIM_IN_REFEITORIO: long (nullable = true)
-- ID_DIM_IN_COMPUTADOR: long (nullable = true)
-- ID_DIM_IN_INTERNET: long (nullable = true)
-- ID_DIM_IN_EQUIP_NENHUM: long (nullable = true)
```

```
In [26]: facts_data_casted\
        .write\
        .jdbc(
            **POSTGRES_CONFIG,
            table=FACT_TABLE_NAME,
            mode="append"
        )

print(f"[{datetime.now()}] ✅ Facts table written to PostgreSQL!")
print("🚀 ETL Pipeline Complete!")
```

[2026-05-12 21:35:37.863229] ✅ Facts table written to PostgreSQL!

🚀 ETL Pipeline Complete!

```
In [28]: import pandas as pd
import matplotlib.pyplot as plt
import psycpg2

conn2 = psycpg2.connect(
    host="127.0.0.1",
    port=5432,
    dbname="censo_escolar",
    user="censo",
    password="123"
)
print("✅ Connected!")
```

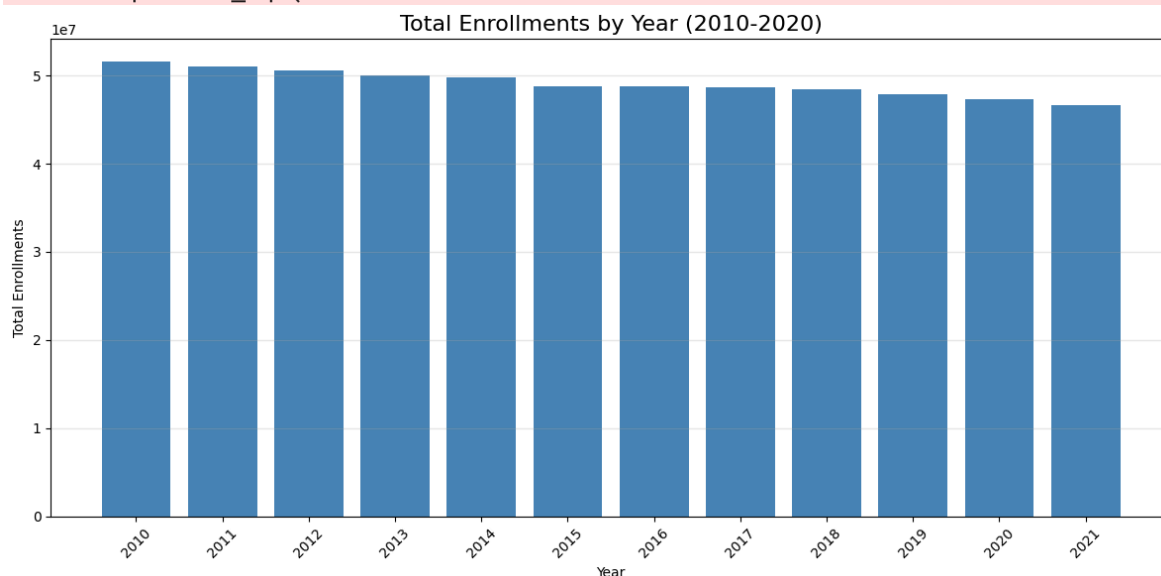
```
# Chart 1 - Enrollments by Year
df1 = pd.read_sql("""
    SELECT NU_ANO_CENSO as year,
           SUM(QT_MAT_BAS) as total_enrollments
    FROM fact_censo_escolar
    GROUP BY NU_ANO_CENSO
    ORDER BY NU_ANO_CENSO
""", conn2)

plt.figure(figsize=(12,6))
plt.bar(df1['year'], df1['total_enrollments'], color='steelblue')
plt.title('Total Enrollments by Year (2010-2020)', fontsize=16)
plt.xlabel('Year')
plt.ylabel('Total Enrollments')
plt.xticks(df1['year'], rotation=45)
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.savefig('chart1_enrollments_by_year.png', dpi=150)
plt.show()
print("✅ Chart 1 done!")
```

✅ Connected!

C:\Users\HUAWEI\AppData\Local\Temp\ipykernel_3328\1411669885.py:15: UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider using SQLAlchemy.

```
df1 = pd.read_sql("""
```



✅ Chart 1 done!

```
In [32]: # Check actual table names in database
cursor = conn2.cursor()
cursor.execute("""
    SELECT table_name
    FROM information_schema.tables
    WHERE table_schema = 'public'
    ORDER BY table_name
""")
tables = cursor.fetchall()
print("All tables:")
for t in tables:
```

```
print("-", t[0])
cursor.close()
```

All tables:

- dim_in_agua_potavel
- dim_in_banheiro
- dim_in_biblioteca
- dim_in_computador
- dim_in_energia_inexistente
- dim_in_equip_nenhum
- dim_in_esgoto_inexistente
- dim_in_internet
- dim_in_refeitorio
- dim_local
- dim_tp_dependencia
- dim_tp_localizacao
- fact_censo_escolar

In [25]: `from sqlalchemy import create_engine`

```
engine = create_engine("postgresql://censo:123@127.0.0.1:5432/censo_escolar")
print("✅ SQLAlchemy engine created!")
```

✅ SQLAlchemy engine created!

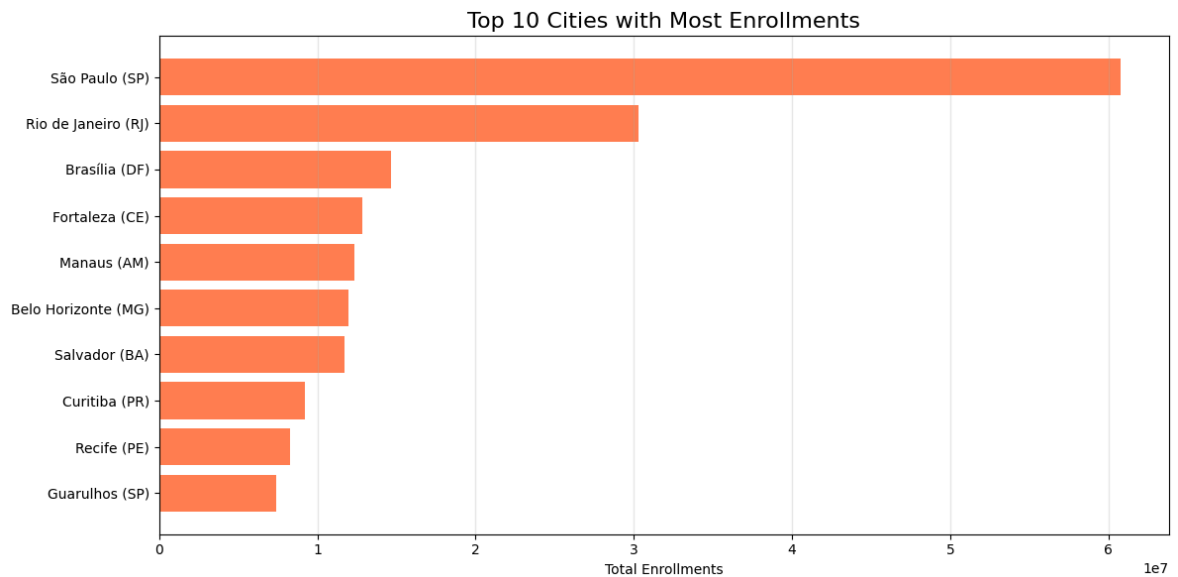
In [34]: `# Chart 2 - Top 10 Cities (Fixed Case Sensitivity)`

```
df2 = pd.read_sql("""
    SELECT l."NO_MUNICIPIO" as city,
           l."SG_UF" as state,
           SUM(f.qt_mat_bas) as total_enrollments
    FROM fact_censo_escolar f
    JOIN dim_local l ON f.id_dim_local = l.id
    GROUP BY l."NO_MUNICIPIO", l."SG_UF"
    ORDER BY total_enrollments DESC
    LIMIT 10
""", engine)

plt.figure(figsize=(12,6))
# Using .astype(str) to ensure concatenation works smoothly
plt.barh(df2['city'].astype(str) + ' (' + df2['state'].astype(str) + ')',
         df2['total_enrollments'], color='coral')

plt.title('Top 10 Cities with Most Enrollments', fontsize=16)
plt.xlabel('Total Enrollments')
plt.gca().invert_yaxis()
plt.grid(axis='x', alpha=0.3)
plt.tight_layout()
plt.savefig('chart2_top10_cities.png', dpi=150)
plt.show()

print("✅ Chart 2 done!")
```



✅ Chart 2 done!

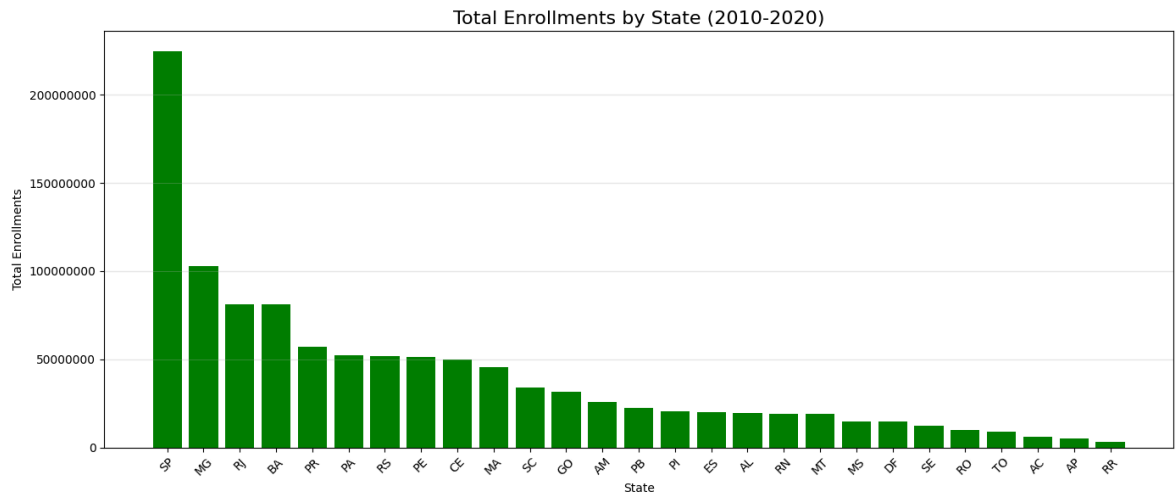
```
In [35]: # Chart 3 - Enrollments by State (Fixed Case Sensitivity)
df3 = pd.read_sql("""
    SELECT l."SG_UF" as state,
           SUM(f.qt_mat_bas) as total_enrollments
    FROM fact_censo_escolar f
    JOIN dim_local l ON f.id_dim_local = l.id
    GROUP BY l."SG_UF"
    ORDER BY total_enrollments DESC
""", engine)

plt.figure(figsize=(14,6))
plt.bar(df3['state'], df3['total_enrollments'], color='green')
plt.title('Total Enrollments by State (2010-2020)', fontsize=16)
plt.xlabel('State')
plt.ylabel('Total Enrollments')

# Format y-axis to show millions clearly if needed
plt.ticklabel_format(style='plain', axis='y')

plt.xticks(rotation=45)
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.savefig('chart3_by_state.png', dpi=150)
plt.show()

print("✅ Chart 3 done!")
print("🎉 All charts completed!")
```



✅ Chart 3 done!

🎉 All charts completed!

References

<https://towardsdatascience.com/explaining-technical-stuff-in-a-non-technical-way-apache-spark-274d6c9f70e9>

<https://towardsdatascience.com/adding-sequential-ids-to-a-spark-dataframe-fa0df5566ff6>

<https://sparkbyexamples.com/pyspark/pyspark-read-and-write-parquet-file/>

<https://www.psycopg.org/docs/usage.html>